Perangkat Lunak Matematika

Tugas 2

Aturan Pengerjaan:

- Tuliskan identitas Anda dengan urutan: Nama, NRP.
- Telusuri program yang Anda buat dan jelaskan output yang dihasilkan pada program yang dibuat.
- Kirimkan file jawaban Anda dalam ekstensi .pdf dengan format nama "Tugas 2-PLM-Nama" paling lambat sebelum kelas dimulai. Keterlambatan pengumpulan jawaban akan berakibat pada penalti nilai. Pengumpulan file jawaban yang terlambat lebih dari satu hari akan dianggap tidak mengerjakan.

In [4]:
```
# Tuliskan library yang Anda perlukan disini
import cmath
from sympy import symbols, diff, sympify, digamma, polygamma
```

Nomor 1

Buatlah fungsi yang dapat mengeluarkan output berupa akar dari persamaan kuadrat

$$ax^2 + bx + c = 0$$

dengan input a, b, dan c berupa float. Berikan contoh untuk kasus akar real dan akar kompleks.

In [13]:
```
## Tuliskan jawaban Anda disini
def akar_fungsi(a, b, c):
    D = b**2 - 4 * a * c

    x1 = (-b + cmath.sqrt(D)) / (2 * a)
    x2 = (-b - cmath.sqrt(D)) / (2 * a)

    return x1, x2
```

In [14]:
```
# 1, -3 , 2 (akar real)
a = float(input("Masukkan a: "))
b = float(input("Masukkan b: "))
c = float(input("Masukkan c: "))

real = akar_fungsi(a, b, c)
print(
    f"Nilai akar-akar dari {a}^2x {"+" if b > 0 else "-"} {abs(b)}x {"+" if c > 0 else "-"} {abs(c)} = 0 adalah {real
)
```

```
Nilai akar-akar dari 1.0^2x - 3.0x + 2.0 = 0 adalah ((2+0j), (1+0j))
```

In [15]:
```
# 1, 2, 5 (akar kompleks)
a = float(input("Masukkan a: "))
b = float(input("Masukkan b: "))
c = float(input("Masukkan c: "))

kompleks = akar_fungsi(a, b, c)
print(
    f"Nilai akar-akar dari {a}^2x {"+" if b > 0 else "-"} {abs(b)}x {"+" if c > 0 else "-"} {abs(c)} = 0 adalah {komp
)
```

```
Nilai akar-akar dari 1.0^2x + 2.0x + 5.0 = 0 adalah ((-1+2j), (-1-2j))
```

Nomor 2

Buatlah fungsi yang dapat mengeluarkan output berupa tabel daftar nilai mata kuliah Perangkat Lunak yang terdiri dari:

- Nama mahasiswa (minimum terdiri atas 4 mahasiswa);
- Nilai Tugas Individu;
- Nilai Tugas Kelompok;
- Nilai ETS;
- Nilai EAS;
- Nilai Akhir, dengan ketentuan 20% dari Tugas Individu, 20% dari Tugas Kelompok, 30% dari ETS, dan 30% dari EAS.

In [16]:
```
## Tuliskan jawaban Anda disini
def nilai_mahasiswa(nama, nrp, individu, kelompok, ets, eas):
    akhir = 0.2 * individu + 0.2 * kelompok + 0.3 * ets + 0.3 * eas
```

```
    str = f"""
    Nama Mahasiswa = {nama}
    NRP Mahasiswa  = {nrp}

    Nilai individu = {0.2 * individu} ({individu})
    Nilai kelompok = {0.2 * kelompok} ({kelompok})
    Nilai ETS      = {0.3 * ets} ({ets})
    Nilai EAS      = {0.3 * eas} ({eas})

    Nilai Akhir    = {akhir}
    """

    print(str)
```

In [17]:

```
nama = str(input("Masukkan nama mahasiswa: "))
nrp = str(input("Masukkan NRP mahasiswa: "))

individu = int(input("Masukkan nilai tugas individu: "))
kelompok = int(input("Masukkan nilai tugas kelompok: "))
ets = int(input("Masukkan nilai ETS: "))
eas = int(input("Masukkan nilai tugas EAS: "))

nilai_mahasiswa(nama, nrp, individu, kelompok, ets, eas)
```

```
Nama Mahasiswa = addien
NRP Mahasiswa  = 5002231063

Nilai individu = 16.0 (80)
Nilai kelompok = 14.0 (70)
Nilai ETS      = 27.0 (90)
Nilai EAS      = 27.0 (90)

Nilai Akhir    = 84.0
```

Nomor 3

Buatlah fungsi yang dapat mengeluarkan output berupa nilai minimum dari suatu fungsi yang didefinisikan pada interval tertentu dengan metode Biseksi. Berikan 2 contoh kasus.

Petunjuk: Ingat kembali metode Biseksi pada Metode Numerik.

Dasar Metode Biseksi

Langkah utama metode Biseksi adalah membagi interval menjadi dua bagian dan menentukan sub-interval yang masih mengandung akar.

Diberikan interval awal ([a,b]), titik tengah interval dihitung dengan

$$c = \frac{a+b}{2}$$

Kemudian nilai fungsi pada titik tersebut dievaluasi.

Jika

$$f(a) \cdot f(b) < 0$$

maka akar berada pada interval

$$[a, b]$$

Sebaliknya jika

$$f(c) \cdot f(b) < 0$$

maka akar berada pada interval

$$[c, b]$$

Proses ini diulangi dengan mengganti interval awal menggunakan interval baru yang masih mengandung akar. Iterasi metode Biseksi dihentikan ketika salah satu kondisi berikut terpenuhi:

1. Selisih antara dua iterasi berturut-turut sangat kecil

$$|x_{n+1} - x_n| < \epsilon$$

2. Nilai fungsi sudah sangat mendekati nol

$$|f(x_n)| < \epsilon$$

3. Iterasi telah mencapai jumlah maksimum yang ditentukan.

In [5]:
```
def biseksi(i, fx, a, b):

    toleransi = 1e-6

    x = symbols("x")
    c = (a + b) / 2

    df = diff(sympify(fx), x)
    fx_a = df.subs(x, a)
    fx_b = df.subs(x, b)
    fx_c = df.subs(x, c)

    print(
            f"i: {i} - a: {sympify(a).evalf():.6f} - b: {sympify(b).evalf():.6f} - fx(a): {fx_a:.6f} - fx(b): {fx_b:
        )

    if abs(b - a) < toleransi:
        nilai_min = sympify(fx).subs(x, c)
        res = f"""
        Nilai x minimum adalah {sympify(c).evalf():.6f}
        Nilai dari fx({sympify(c).evalf():.6f}) = {nilai_min.evalf():.6f}
        """

        print(res)
        return
    elif fx_a * fx_c < 0:
        biseksi(i + 1, fx, a, c)
    elif fx_b * fx_c < 0:
        biseksi(i + 1, fx, c, b)
```

In [6]:
```
fx = input("Masukkan fungsi: ")
a = int(input("Masukkan nilai awal:"))
b = int(input("Masukkan nilai akhir:"))
```

```python
print(f"fungsi: {fx} dengan interval [{a}, {b}]")
biseksi(1, fx, a, b)
```

```
fungsi: x**2 -4 *x + 3 dengan interval [0, 3]
i: 1 - a: 0.000000 - b: 3.000000 - fx(a): -4.000000 - fx(b): 2.000000 - c: 1.500000 - fx(c): -1.000000
i: 2 - a: 1.500000 - b: 3.000000 - fx(a): -1.000000 - fx(b): 2.000000 - c: 2.250000 - fx(c): 0.500000
i: 3 - a: 1.500000 - b: 2.250000 - fx(a): -1.000000 - fx(b): 0.500000 - c: 1.875000 - fx(c): -0.250000
i: 4 - a: 1.875000 - b: 2.250000 - fx(a): -0.250000 - fx(b): 0.500000 - c: 2.062500 - fx(c): 0.125000
i: 5 - a: 1.875000 - b: 2.062500 - fx(a): -0.250000 - fx(b): 0.125000 - c: 1.968750 - fx(c): -0.062500
i: 6 - a: 1.968750 - b: 2.062500 - fx(a): -0.062500 - fx(b): 0.125000 - c: 2.015625 - fx(c): 0.031250
i: 7 - a: 1.968750 - b: 2.015625 - fx(a): -0.062500 - fx(b): 0.031250 - c: 1.992188 - fx(c): -0.015625
i: 8 - a: 1.992188 - b: 2.015625 - fx(a): -0.015625 - fx(b): 0.031250 - c: 2.003906 - fx(c): 0.007812
i: 9 - a: 1.992188 - b: 2.003906 - fx(a): -0.015625 - fx(b): 0.007812 - c: 1.998047 - fx(c): -0.003906
i: 10 - a: 1.998047 - b: 2.003906 - fx(a): -0.003906 - fx(b): 0.007812 - c: 2.000977 - fx(c): 0.001953
i: 11 - a: 1.998047 - b: 2.000977 - fx(a): -0.003906 - fx(b): 0.001953 - c: 1.999512 - fx(c): -0.000977
i: 12 - a: 1.999512 - b: 2.000977 - fx(a): -0.000977 - fx(b): 0.001953 - c: 2.000244 - fx(c): 0.000488
i: 13 - a: 1.999512 - b: 2.000244 - fx(a): -0.000977 - fx(b): 0.000488 - c: 1.999878 - fx(c): -0.000244
i: 14 - a: 1.999878 - b: 2.000244 - fx(a): -0.000244 - fx(b): 0.000488 - c: 2.000061 - fx(c): 0.000122
i: 15 - a: 1.999878 - b: 2.000061 - fx(a): -0.000244 - fx(b): 0.000122 - c: 1.999969 - fx(c): -0.000061
i: 16 - a: 1.999969 - b: 2.000061 - fx(a): -0.000061 - fx(b): 0.000122 - c: 2.000015 - fx(c): 0.000031
i: 17 - a: 1.999969 - b: 2.000015 - fx(a): -0.000061 - fx(b): 0.000031 - c: 1.999992 - fx(c): -0.000015
i: 18 - a: 1.999992 - b: 2.000015 - fx(a): -0.000015 - fx(b): 0.000031 - c: 2.000004 - fx(c): 0.000008
i: 19 - a: 1.999992 - b: 2.000004 - fx(a): -0.000015 - fx(b): 0.000008 - c: 1.999998 - fx(c): -0.000004
i: 20 - a: 1.999998 - b: 2.000004 - fx(a): -0.000004 - fx(b): 0.000008 - c: 2.000001 - fx(c): 0.000002
i: 21 - a: 1.999998 - b: 2.000001 - fx(a): -0.000004 - fx(b): 0.000002 - c: 2.000000 - fx(c): -0.000001
i: 22 - a: 2.000000 - b: 2.000001 - fx(a): -0.000001 - fx(b): 0.000002 - c: 2.000000 - fx(c): 0.000000
i: 23 - a: 2.000000 - b: 2.000000 - fx(a): -0.000001 - fx(b): 0.000000 - c: 2.000000 - fx(c): -0.000000

        Nilai x minimum adalah 2.000000
        Nilai dari fx(2.000000) = -1.000000
```

Nomor 4

Buatlah fungsi yang dapat mengeluarkan output berupa nilai minimum dari suatu fungsi yang diberikan dengan metode Newton-Raphson. Berikan 3 contoh kasus, dengan salah satu contoh berupa kasus dimana metode Newton-Raphson tidak menghasilkan solusi.

Petunjuk:

- Ingat kembali metode Newton-Raphson pada Metode Numerik.
- Jumlah iterasi maksimum default adalah 100.

Metode Newton–Raphson

Metode Newton–Raphson merupakan metode numerik yang digunakan untuk mencari akar dari suatu persamaan non-linear

$$f(x) = 0$$

Jika diberikan suatu tebakan awal (x_0), maka pendekatan akar selanjutnya dihitung menggunakan rumus iterasi:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

di mana:

- (x_n) adalah pendekatan pada iterasi ke-(n)
- ($f(x_n)$) adalah nilai fungsi pada titik (x_n)
- ($f'(x_n)$) adalah turunan pertama fungsi pada titik (x_n)

Metode ini diulangi hingga nilai (x_n) konvergen ke akar yang diinginkan. Iterasi metode Newton–Raphson dihentikan apabila salah satu kondisi berikut terpenuhi:

1. Perubahan antara dua iterasi sangat kecil

$$|x_{n+1} - x_n| < \epsilon$$

2. Nilai fungsi sudah mendekati nol

$$|f(x_n)| < \epsilon$$

3. Iterasi telah mencapai jumlah maksimum yang ditentukan.

In [11]:
```
def newton_raph(i, fx, x_n):
    if i > 100:
        print("iterasi melebihi batas maksimum")
```

```
        return

    toleransi = 1e-6
    try:
        x = symbols("x")
        f = sympify(fx)
        df = diff(f, x)
        gx = diff(df, x)

        fx_subs = df.subs(x, x_n)
        gx_subs = gx.subs(x, x_n)

        x_np1 = x_n - (fx_subs / gx_subs)

        print(
            f"i: {i} - x_n: {sympify(x_n).evalf():.6f} - f(x): {fx_subs.evalf():.6f} - f'(x): {gx_subs.evalf():.6f} -
        )

        if abs(x_np1 - x_n) < toleransi:
            x_min = x_np1.evalf()
            f_xmin = f.subs(x, x_min).evalf()

            print(f"Nilai minimum didapat = {x_min:.6f}")
            print(f"Nilai dari f(x_min) = {f_xmin:.6f}")

            return
        else:
            newton_raph(i + 1, fx, x_np1)
    except ZeroDivisionError:
        print("Nilai pembagi tidak boleh = 0")
        return
```

In [12]:

```
fx = input("Masukkan fungsi: ")
x_n = int(input("Masukkan nilai awal: "))

print(f"fungsi: {fx} dengan interval [{a}, {b}]")
newton_raph(1, fx, x_n)
```

```
fungsi: x**2 - 4 * x + 3 dengan interval [0, 3]
i: 1 - x_n: 0.000000 - f(x): -4.000000 - f'(x): 2.000000 - x_np1: 2.000000
i: 2 - x_n: 2.000000 - f(x): 0.000000 - f'(x): 2.000000 - x_np1: 2.000000
Nilai minimum didapat = 2.000000
Nilai dari f(x_min) = -1.000000
```

Nomor 5

Diketahui suatu data sebagai berikut.

8	6	4	9
6	14	14	7
12	18	7	6
6	6	9	20
10	15	7	15
21	16	8	4
9	9	15	6
9	5	10	9
12	6	2	12
8	10	13	3
4	9	4	6
13	5	7	10
15	5	11	10
15	9	9	11
12	11	9	15
6	6	15	5
2	12	6	10

5	12	8	3
6	15	16	8
5	7	2	6
7	10	15	8
13	14	3	13
10	15	5	13
13	11	5	16
14	5	17	21

Diasumsikan data tersebut mengikuti distribusi Negatif Binomial dengan Fungsi Massa Probabilitas (PMF)

$$p(X = x|p, r) = \binom{x + r - 1}{r - 1} p^x (1 - p)^r, \quad 0 < p < 1, r = 1, 2, \ldots$$

1. Dapatkan fungsi log-likelihood, vektor gradien, dan matriks Hessian dari distribusi Negatif Binomial.
2. Buatlah fungsi yang dapat mengeluarkan output berupa estimator Maximum Likelihood parameter p dan r dari data tersebut dengan menggunakan metode Newton-Raphson.

Petunjuk:

- Ingat kembali materi estimasi parameter menggunakan MLE di Matematika Statistika.
- Gunakan input berupa nilai awal p, r, dan jumlah iterasi maksimum sebanyak 100.

Log-Likelihood

Jika terdapat data pengamatan $x_1, x_2, \ldots, x_n$ maka likelihood dari seluruh data adalah

$$L(p, r) = \prod_{i=1}^{n} \binom{x_i + r - 1}{r - 1} p^{x_i} (1 - p)^r$$

Untuk mempermudah perhitungan digunakan **log-likelihood**

$$\ell(p, r) = \log L(p, r)$$

sehingga diperoleh

$$\ell(p, r) = \sum_{i=1}^{n} \left[\log \Gamma(x_i + r) - \log \Gamma(r) - \log \Gamma(x_i + 1) - x_i \log p - r \log(1 - p)\right]$$

Vektor Gradien

Untuk mencari maksimum dari log-likelihood digunakan turunan pertama terhadap parameter. Vektor gradien didefinisikan sebagai

$$\nabla \ell(p, r) = \begin{bmatrix} \frac{\partial \ell}{\partial p} \\ \frac{\partial \ell}{\partial r} \end{bmatrix}$$

Komponen pertama adalah turunan terhadap p:

$$\frac{\partial \ell}{\partial p} = \sum_{i=1}^{n} \frac{x_i}{p} - \frac{nr}{1 - p}$$

Komponen kedua adalah turunan terhadap r:

$$\frac{\partial \ell}{\partial r} = \sum_{i=1}^{n} \left(\psi(x_i + r) - \psi(r)\right) + n \log(1 - p)$$

di mana $\psi(x)$ adalah **fungsi digamma**, yaitu turunan dari log gamma.

Matriks Hessian

$$H(p, r) = \begin{bmatrix} \frac{\partial^2 \ell}{\partial p^2} & \frac{\partial^2 \ell}{\partial p \partial r} \\ \frac{\partial^2 \ell}{\partial r \partial p} & \frac{\partial^2 \ell}{\partial r^2} \end{bmatrix}$$

Elemen-elemen matriks Hessian adalah

$$\frac{\partial^2 \ell}{\partial p^2} = -\sum_{i=1}^{n} \frac{x_i}{p^2} - \frac{nr}{(1-p)^2}$$

$$\frac{\partial^2 \ell}{\partial p \, \partial r} = -\frac{n}{1-p}$$

$$\frac{\partial^2 \ell}{\partial r^2} = \sum_{i=1}^{n} \left(\psi'(x_i + r) - \psi'(r)\right)$$

di mana $\psi'(x)$ adalah **trigamma function**, yaitu turunan dari **digamma function**.

In [18]:
```
## Tuliskan jawaban Anda disini
import math
import numpy as np

data = [
    8, 6, 4, 9,
    6, 14, 14, 7,
    12, 18, 7, 6,
    6, 6, 9, 20,
    10, 15, 7, 15,
    21, 16, 8, 4,
    9, 9, 15, 6,
    9, 5, 10, 9,
    12, 6, 2, 12,
    8, 10, 13, 3,
    4, 9, 4, 6,
    13, 5, 7, 10,
    15, 5, 11, 10,
    15, 9, 9, 11,
    12, 11, 9, 15,
    6, 6, 15, 5,
    2, 12, 6, 10,
    5, 12, 8, 3,
    6, 15, 16, 8,
    5, 7, 2, 6,
    7, 10, 15, 8,
    13, 14, 3, 13,
    10, 15, 5, 13,
    13, 11, 5, 16,
```

```
    14, 5, 17, 21,
]
```

In [19]:

```
def log_likelihood(x_n, p, r):
    res = 0

    for x_i in x_n:
        res += math.lgamma(x_i + r) - math.lgamma(r) - math.lgamma(x_i + 1) + x_i * math.log(p) + r * math.log(1-p)

    return res

def vektor_grad(x_n, p, r):
    dp = 0
    dr = 0

    for x_i in x_n:
        dp += (x_i / p)
        dr += (digamma(x_i + r) - digamma(r))

    dp -= ((len(x_n) * r) / (1 - p))
    dr += (len(x_n) * math.log(1 - p))

    return np.array([
        [dp],
        [dr]
    ])

def matriks_hessian(x_n, p, r):
    dp2 = 0
    dpdr = -len(x_n) / (1 - p)
    drdp = dpdr
    dr2 = 0

    for x_i in x_n:
        dp2 += x_i / (p**2)
        dr2 += polygamma(1, x_i + r) - polygamma(1, r)

    dp2 *= -1
    dp2 -= (len(x_n) * r) / (1 - p)**2

    return np.array([
        [dp2, dpdr],
```

```
        [drdp, dr2.evalf()]
    ])
```

In [ ]:

```
p = float(input("Masukkan nilai p: "))
r = int(input("Masukkan nilai r: "))

ll = log_likelihood(data, p, r)
vek_grad = vektor_grad(data, p, r)
matriks_hess = matriks_hessian(data, p, r)

total_data = len(data)
sum_of_data = sum(i for i in data)

res = f"""
p = {p}
r = {r}
{"-"*20}
total data  = {total_data}
jumalh data = {sum_of_data}
{"-"*20}
- Log-likelihood  = {ll}
- Vektor Gradient
{vek_grad}

- Matriks Hessian
{matriks_hess}
"""

print(res)
```

```
p           = 0.35
r           = 6
--------------------
total data  = 100
jumalh data = 959
--------------------
- Log-likelihood  = -514.4697153414941
- Vektor Gradient
[[1816.9230769230765]
 [53.4022457748045]]

- Matriks Hessian
[[-9248.689771766693 -153.84615384615384]
 [-153.84615384615384 -10.9008249349129]]
```

Nomor 6

Buatlah fungsi yang dapat membangkitkan bilangan berdistribusi Gompertz dari bilangan berdistribusi Uniform-$(0, 1)$ sebanyak n bilangan menggunakan metode Invers Transform jika diketahui CDF Distribusi Gompertz adalah

$$F(x) = 1 - \exp\left(\eta(e^{bx} - 1)\right).$$

Petunjuk:

- Ingat kembali materi pembangkitan bilangan random di Teknik Simulasi.
- Gunakan nilai parameter $\eta = 2$ dan $b = 1.5$ sebagai input default.

Distribusi Gompertz

Distribusi Gompertz memiliki dua parameter yaitu:

- $(\eta > 0)$ → parameter **skala**
- $(b > 0)$ → parameter **shape (tingkat pertumbuhan)**

Fungsi distribusi kumulatif (CDF) dari distribusi Gompertz diberikan oleh:

$$F(x) = 1 - \exp\left(-\eta(e^{bx} - 1)\right), \quad x \geq 0$$

Untuk membangkitkan bilangan acak dari distribusi Gompertz digunakan **metode Inverse Transform**. Jika (U) merupakan bilangan acak dari distribusi Uniform(0,1), maka variabel acak (X) dari distribusi tertentu dapat diperoleh dengan:

$$X = F^{-1}(U)$$

Menentukan Invers CDF:

$$\begin{aligned}
U &= F(x) \\
U &= 1 - \exp\left(-\eta\left(e^{bx} - 1\right)\right) \\
1 - U &= \exp\left(-\eta\left(e^{bx} - 1\right)\right) \\
\ln(1 - U) &= -\eta\left(e^{bx} - 1\right) \\
-\frac{1}{\eta}\ln(1 - U) &= e^{bx} - 1 \\
e^{bx} &= 1 - \frac{1}{\eta}\ln(1 - U) \\
x &= \frac{1}{b}\ln\left(1 - \frac{1}{\eta}\ln(1 - U)\right)
\end{aligned}$$

In [ ]:

```
## Tuliskan jawaban Anda disini

def dist_gompertz(n, eta, b):
    Un = np.random.uniform(0, 1, n)
    res = [(1 / b) * math.log(1 - (math.log(1 - Ui) / eta)) for Ui in Un]
    return res

n = int(input("Masukkan jumlah bilangan random: ") or '1')
b = float(input("Masukkan nilai b (default: 1.5): ") or '1.5')
eta = int(input("Masukkan nilai (default: 2): ") or '2')

rand = dist_gompertz(n, eta, b)
print(rand)
```

```
[0.33468566240628067, 0.02030512625368524, 0.6038146507231288, 0.14350656529539893, 0.1223629756710571, 0.62910933065
97998, 0.45028130064718463, 0.8886673251671227, 0.06543903129743556, 0.008710223680841294]
```